Beyond Programming Languages

Alexander Vityaz

Corezoid Inc., Dnipro, Ukraine
alexander.vityaz@gmail.com
corezoid.com · simulator.company
ORCID: 0009-0006-0489-7881
DOI: 10.5281/zenodo.21458098
Gemini Notebook

Abstract

Programming languages historically combined two functions: making human intent intelligible and making machine execution precise. This article argues that advances in artificial intelligence allow those functions to be separated. AI can mediate between human language and formal system design, while a universal executable Actor Graph can carry identity, accounting state, causal history, transactional relationships, and deterministic transition rules. Drawing on Winograd, Naur, Brooks, model-driven architecture, executable UML, and workflow systems, the article distinguishes the proposal from earlier model-based approaches through global actor identity, a universal actor contract, actor-valued edges, transaction primacy, and recursive composition. It also emphasizes that execution precision does not establish model correctness and that formal categories require provenance, governance, and procedures for challenge. In this account, code becomes a regenerable implementation artifact, while the verified Actor Graph becomes the system's source of operational truth.

Keywords: artificial intelligence; programming languages; Actor Model; Actor Graph; executable models; model-driven engineering; workflow systems; software architecture; formal verification

> *"Bad programmers worry about the code. Good programmers worry about data structures and their relationships."* — *Linus Torvalds*

1. Programming Language as a Historical Intermediary

Programming languages appear inseparable from computing. We are accustomed to treating programming as the writing of text in Python, Java, C++, or another formal language. Syntax, libraries, and development environments change, yet the underlying arrangement seems fixed: a human writes a program; a machine executes it.

A programming language, however, is not a necessary component of a computing machine. A processor does not need Python or Java. It executes machine instructions. Programming languages emerged as intermediaries between human intent and machine execution.

The first programmers worked directly with machine code. This form of programming was extraordinarily slow and expensive. It required people to think in terms of memory addresses, registers, and the instruction set of a particular processor.

Assembly language relieved part of this burden. High-level languages took the next step: they allowed people to describe computations in a more intelligible form, reduced a program's dependence on a particular machine, and delegated translation into machine instructions to the compiler. Development speed increased by orders of magnitude. Modern software products, the software industry, and the profession of the programmer as we know it emerged alongside that shift.

Linus Torvalds summarized this historical sequence as follows:

> *"And AI will increase your productivity by a factor of 10. And I claim that compilers increase your productivity by a factor of a thousand."*
>
> *— Linus Torvalds, Open Source Summit North America, 20 May 2026 [6]*

Immediately before this statement, Torvalds traced the path from writing machine instructions directly, through assembly and the compiler, to AI tools. His point is clear: AI extends a long line of tools, each removing another layer of manual work from the human programmer while preserving the basic structure of programming itself.

For this argument, the proposed acceleration factors are secondary. The sequence itself matters: machine code → assembly language → high-level language → AI tools. Torvalds places AI at the end of this chain as another force multiplier for the programmer.

Yet the question of the chain's own limits was raised much earlier.

2. Winograd's Question

In July 1979, *Communications of the ACM* published Terry Winograd's *Beyond Programming Languages*. Winograd argued that existing concepts of programming languages were logically sufficient but practically insufficient for constructing and maintaining systems of increasing complexity. The next level, he proposed, should shift attention away from specifying algorithms in detail and toward the properties of the packages, objects, and systems with which the programmer works [1].

This was no mere prophecy. Winograd proposed higher-level programming systems, rich interactive environments, movement among different representations of the same system, progressive refinement from the general to the particular, and support for integrating, modifying, and explaining existing programs. Later, with Fernando Flores, he applied the language/action perspective to organizational interaction, and ActionWorkflow turned commitments, roles, and the permitted moves of participants into an executable workflow system.

The distinction, therefore, cannot be framed as though Winograd had no view of what might follow programming languages. He saw the direction with considerable accuracy. What did not yet exist was the combination of two mechanisms: a machine capable of extracting the structure of intent from natural language, documents, examples, and data; and a universal executable model capable of preserving a system's identity, accounting state, and causal history independently of any particular implementation.

This gives us three positions. Torvalds treats AI as a new link in the chain of translation. Winograd questioned the adequacy of the language-centered arrangement and began building systems at a higher level. This essay proposes the next step: separating the functions that programming languages were historically forced to combine, and removing the programming language as a mandatory human-facing layer.

To determine whether this is possible, we must first identify what a programming language has actually done.

3. The Two Functions of a Programming Language

In the relationship between human and machine, programming languages have always performed two primary functions.

The first is **intelligibility**. A person must be able to express an intention, read the resulting program, discuss it with others, and modify it.

The second is **precision**. A machine does not understand approximate instructions. Every condition, value, and action must be defined unambiguously. A program either permits a transition to the next state or it does not. It either executes a transaction or rejects it.

For a long time, a programming language was the only practical means of satisfying both functions at once. It was formal enough for the machine and intelligible enough for the human. This is why source code became the central artifact of development and the source of truth about the system.

That coupling came at a price. Humans had to think in a form that could be translated into machine execution without loss. The machine, in turn, received not the model of reality itself, but the implementation text of that model.

AI dissolves the historical necessity of keeping intelligibility and precision in a single artifact.

4. Why Code Does Not Contain the Whole System

The cost of a code-centered arrangement becomes especially visible as a system ages. Every substantial change begins with archaeology: the team must reconstruct how code, data, configuration, database schemas, and integrations interact; which undocumented exceptions have become operating rules; and which assumptions made by earlier developers are still preventing the system from falling apart.

The more legacy accumulates, the more time is spent not changing the system, but reconstructing the truth about it. The codebase continues to be treated as an asset, while an increasing share of it becomes accumulated uncertainty.

In *Programming as Theory Building*, Peter Naur offered a more precise explanation of this phenomenon. For Naur, the primary product of programming is not the program text, but the theory that programmers construct about how the program's execution solves the problem at hand. Code and documentation are expressions of that theory, but they do not exhaust it. When the team that possesses the theory dissolves, the program may continue to run, while the ability to change it meaningfully declines sharply [2].

This is why a functioning codebase can be both a precise implementation and an incomplete source of truth. A substantial part of the system model remains distributed across code, data, configuration, documentation, the history of decisions, and people's tacit knowledge. Legacy archaeology is an attempt to reconstruct the lost theory from the traces it left behind.

Naur explains why code cannot be a complete model of the system. The next question is what can replace it without sacrificing precision.

5. AI Takes Over the Function of Intelligibility

A person no longer has to learn the syntax of a formal language to explain to a machine what kind of system should be built. The task can be described in ordinary language, illustrated with examples, and supplemented with documents, tables, images, records of real operations, regulatory requirements, and existing data.

AI can infer the system structure implied by this context, propose an architecture, identify recurring rules, expose contradictions, and construct an initial formal model.

This often leads to a premature conclusion: natural language will become the new programming language. A person will speak to AI, and AI will write programs.

Natural language, however, does not solve the problem of precision.

The sentence "approve loans quickly for good customers" is intelligible to a person, but it does not define an operational system. Who counts as a good customer? What does "quickly" mean? Which data are mandatory? Who has the authority to decide? Can the decision be reversed? What happens when sources conflict? How is the basis for a rejection recorded? Which version of the rule was in force at the moment of decision?

A large language model can propose answers, but those answers do not follow logically from the original sentence itself. The probabilistic nature of the model does not necessarily mean that each run will produce different text: output can be made reproducible through technical settings. The problem lies deeper. There is no guaranteed formal correspondence between a human formulation and a specific rule. The model proposes an interpretation; it does not prove that this interpretation expresses the system owner's will.

Using an AI response as a direct instruction for a critical operational system transfers the uncertainty of human language into execution.

AI can take over the translation of human intent into a formal model. It does not thereby become the source of precision. The model it proposes must be presented, verified, accepted, and only then admitted to execution.

6. The Actor Graph Becomes the Source of Precision

With the emergence of AI, the two functions of a programming language can be separated.

AI handles interaction with humans. The Actor Graph handles the precise description, verification, and execution of the system.

Here, the Actor Graph refers to more than the classical Actor Model used to organize concurrent and distributed computation. An actor is a unit of the modeled reality itself: it may be a person, an organization, a document, a device, a contract, a form field, a process, or a software component. The graph therefore describes not only the internal organization of computation, but also the system together with its relations to the external environment.

Not every noun appearing in a domain description becomes an actor. Actor status belongs to an operationally significant role: a role with a `global_actor_id`, a distinguishable accounting state, and a participation interface. An action by such a role can change the admissibility of a transaction, its outcome, causal trace, accounting state, right, or risk. If a form field affects whether a transition is permitted or records a legally significant decision, it may be an actor at that scale. If it is a causally neutral detail of the interface, it remains part of the surrounding actor's process. The boundary depends on scale, but the criterion remains strict [7].

The minimal contract of every actor is:

$$\text{Actor} = \langle \texttt{global\_actor\_id}, \texttt{accounts}, \texttt{events}, \texttt{process} \rangle .$$

global_actor_id establishes the actor's identity. It is neither a local address for message delivery nor the identifier of a record inside a particular application. It belongs to the actor itself and persists across changes of runtime, application, process, implementation, and representational scale.

accounts define the actor's accounting state and transactional memory: account trees, plan/fact and debit/credit structures, and other typed states associated with its global identity. These are not limited to monetary accounts. The system may account for statuses, quantitative indicators, obligations, rights, limits, versions, and any other typed states.

events preserve external and internal events, causal order, time, trace, and evidence. They connect the current state to the way in which it arose.

process defines the rules for processing transactions and transitions. It may be an ordinary process, another actor, or a nested Actor Graph:

$$\text{process}(a) \in \{\text{Process, Actor, ActorGraph}\}.$$

The `global_actor_id` provides the identity foundation of the graph. The same actor can participate in multiple subgraphs and contexts without becoming a new proxy entity each time. A collapsed graph can be represented as a single actor and later expanded into a nested Actor Graph; its internal actors retain their identities and transactional memory.

The basic construction is:

$$\texttt{global\_actor\_id} \longrightarrow \text{transactions} \longrightarrow \text{Actor Graph}.$$

A unified space of globally identified actors comes first. Transactions then connect them. The topology of the graph arises neither from a drawing nor from the adjacency of records in a database, but from a system of transactional relations among persistently identified participants.

7. The Edge Is Also an Actor

In an ordinary diagram, a relationship is drawn as a line between two nodes. Such a line appears passive: it has no memory, behavior, rights, or history of its own. In a real system, however, a substantial share of the complexity often resides inside the relationship itself.

Torvalds's epigraph captures half of this shift: good programmers look beyond code to data structures and their relationships. Yet the formulation stops at the relationship as a connection between structures. The Actor Graph takes the next step: when a relationship has its own identity, `accounts`, `events`, and `process`, it is itself a structure—an autonomous edge actor.

A payment between a customer and a merchant has its own stages, fees, obligations, timeouts, reversals, disputes, and evidence. A delivery between a customer and a supplier has a route, deadlines, documents, responsibility, and a fulfillment state. A contract, a delegation of authority, a grant of access, or an AI decision may likewise have its own identity and lifecycle.

An edge in the Actor Graph is therefore not an attribute of an adjacent node, but a full-fledged **edge actor** with the same `global_actor_id/accounts/events/process` contract. When necessary, its process expands into a nested graph.

This distinction is substantive. If collapsing a relationship into a passive line changes the outcome, trace, ledger, right, risk, attribution, authority, or provenance, an edge actor is semantically

necessary. Reducing such a relationship to a field in a record destroys part of the system, which must later be reconstructed from logs, agreements, and human explanations.

A vertex, an edge, an event, a trigger, an external participant, and a nested graph are not different ontological species. They are roles and configurations of one universal type: Actor.

8. A Transaction Creates the Graph

In the classical Actor Model, the primary unit of interaction is the message. In the proposed construction, any message recognized by the graph becomes a transaction. Before registration, a raw external signal may remain a signal or noise; a transaction begins where the system assigns operational significance to a difference.

A transaction contains, at minimum:

- its own `transaction_id`;
- `source_actor_id`, `edge_actor_id`, and `destination_actor_id`;
- a `currency/value` pair;
- a protocol, preconditions, metadata, and effects.

Here, `currency` is an open type for value that is accounted for, transferred, or changed; it is not limited to money. It may be a number, a monetary amount, a date and time, a coordinate, a status, a document, a right, or the `global_actor_id` of another participant. `value` belongs to the domain of the selected `currency`.

A transaction thus connects identities, changes `accounts`, generates `events`, and invokes `process` at the same time. Cancellations, reversals, and compensations are represented as new transactions rather than by rewriting the past. The graph therefore preserves both the current state and the immutable causal history of how that state arose [8].

The Actor Graph defines the structure of the system itself rather than a text of commands:

- which actors exist and how their global identities persist;
- which `accounts` constitute their accounting state;
- which `events` have causal significance;
- which `processes` permit transitions;
- which relationships must be represented as edge actors;
- which transactions are permitted and which preconditions are mandatory;
- who has the authority to make decisions and recognize an outcome;
- where the boundaries of responsibility and jurisdiction lie;
- which changes are prohibited;
- how an outcome is connected to its trace and ledger.

Once the graph has been constructed, verified, and accepted, execution no longer depends on a probabilistic interpretation of human text. The graph either permits a particular transition or it does not. A transaction either satisfies the established contract or is rejected.

9. Precision Changes Its Carrier

Determinism is not an exclusive property of the graph. A program written in a formal language also executes according to defined rules. What changes is not the existence of precision, but its carrier.

In a traditional system, precision is fixed in the implementation, while the model of the system itself is distributed across source code, the database schema, configuration, documentation, and the knowledge of developers. In the graph, global identities, `accounts`, `events`, `processes`, transactions, rights, boundaries, and relationships are represented directly in a single executable model.

The shift is not from imprecise code to a precise graph. Precision moves from the text of the implementation into the model of the system.

AI can create the graph, extend it, detect contradictions, and propose changes. Once those changes have been accepted, however, they are executed by a deterministic environment rather than by a language model.

A universal Actor type does not imply universal control. An external person, bank, organization, or device may operate under another jurisdiction and retain its own authority over execution. The graph does not acquire magical control over the external world. It preserves the participant's global identity, interaction protocol, evidence, declared state, deadlines, and rules for recognizing the result. An external actor may be outside a particular runtime, but it is not outside the model.

Precision of execution must be distinguished from correctness of the model. **A graph can execute an incorrectly specified system with perfect precision.** Verification must therefore include inspection of the graph's structure and invariants, detection of inadmissible states, simulation, reproduction of real-world scenarios, replay of historical transactions, verification of rights, and, for critical properties, formal proof.

The graph provides precision of execution. Whether the graph corresponds to human intent and to reality remains a separate engineering problem.

The future chain for creating systems therefore changes from:

$$\text{intent} \longrightarrow \text{AI} \longrightarrow \text{programming language} \longrightarrow \text{compiler} \longrightarrow \text{machine code}$$

to:

$$\text{intent} \longrightarrow \text{AI} \longrightarrow \text{Actor Graph} \longrightarrow \text{verification} \longrightarrow \text{execution}.$$

Execution returns events, transactions, and telemetry to the same graph, closing the loop of observation and change.

The Actor Graph becomes the source of **operational truth for the system**: the authoritative model of the identities, states, events, transitions, and evidence that the system recognizes. This does not make the graph a source of truth about the world. AI becomes the interface through which people work with the accepted model.

10. Why Earlier Model-Based Approaches Did Not Replace Code

The idea of making the model primary has serious predecessors. CASE, Model-Driven Architecture, and executable UML raised the level of abstraction, separated the model from the platform, and made it possible to derive, run, and test an implementation from a model. The distinction

introduced here therefore cannot be reduced to an opposition between a static diagram and an executable model [4, 5].

The closest predecessor was Winograd himself. In the late 1980s, he and Fernando Flores developed the language/action perspective, applied it in *The Coordinator*, and subsequently created ActionWorkflow—a methodology and software architecture in which a business process was represented as a network of commitment cycles between a customer and a performer [9, 10, 11]. ActionWorkflow supported roles, conditions of satisfaction, stages from request to acceptance of the result, branching and parallelism, cost/value, a history of workflow transactions, an execution server, APIs, and application generation; its patents formalized the permitted moves of participants and the assembly of applications from connected workflows [12].

In the 1990s, the Workflow Management Coalition standardized a reference architecture for workflow systems and interfaces for process definitions, client applications, invoked applications, interoperability, administration, and monitoring [13]. The BPMN standard later established a common notation for business processes, while BPMN 2.0.2 introduced a normative machine-readable metamodel and schemas. Workflow thereby developed from the architecture of individual products into an industrially standardized layer [14].

The Actor Graph therefore cannot claim to have invented the executable model of an organization, typed interactions, workflow history, the workflow server, or application generation. ActionWorkflow is a direct architectural predecessor for this class of problem. This is a retrospective comparison, not a claim of a direct genealogy of influence.

The workflow model developed by Winograd and Flores can be understood as an important class of **edge actor**: a relationship within which roles, commitments, events, conditions, and execution exist. The distinction between that approach and the Actor Graph rests on four invariants.

Table 1: Comparison of ActionWorkflow and the Actor Graph

Invariant	ActionWorkflow and standardized workflow	Actor Graph
Identity	A role is identified within a workflow system or process.	The `global_actor_id` belongs to the actor and persists across runtimes, applications, implementations, and graphs.
Memory	State, cost/value, and history belong to a workflow or case; a full contract for every relationship is not required.	Every actor, including an edge actor, has `accounts`, `events`, and its own transactional memory.
Relationship	Links and flows organize coordination but need not be entities of the same type as the participants.	A relationship may be an edge actor with a full contract and a `process` that expands into a nested graph.
Scope	The primary subject is workflow and the business process.	One recursive type encompasses people, programs, devices, documents, institutions, relationships, and external jurisdictions.

After ActionWorkflow, the novelty claim must be narrow: the combination of global identity, a universal actor contract, actor-valued edges, transaction primacy, and recursive composition within one continuously executing model.

History yields one conclusion. CASE and MDA show how a separate model loses out to the

running system; ActionWorkflow, WfMC, and BPMN show how far executable workflow can advance while remaining an ontology of process. AI changes the cost of constructing the model. The Actor Graph seeks to change the model's status and scope by making the entire system of globally identified actors and their transactional relationships an executable model.

11. Why Programming Languages Need Not Survive

It is sometimes assumed that Python, Java, or C++ will necessarily remain inside the system, while people simply cease to see the code generated by AI. This scenario is possible and, during the transition, almost inevitable. Yet there is no principled need for a human-readable high-level language to remain a mandatory intermediate layer.

A high-level language was historically necessary because humans read, wrote, discussed, and modified it, while the compiler transformed that text into an executable form. When a person works directly with the system model, the graph can be transformed into the environment's internal representation, bytecode, or machine instructions without an intermediate listing in a familiar programming language.

Code will continue to be generated for existing operating systems, libraries, databases, devices, drivers, protocols, and legacy infrastructure for a long time. Its status, however, will change.

Generated code will cease to be the source of truth. It will no longer need to be carefully preserved and modified by hand over many years. It will become a derived artifact that can be deleted and generated again from the graph.

The same model will be able to produce cloud, on-premises, mobile, high-performance, resource-efficient, or special-purpose hardware implementations. Those implementations may reflect different regulatory requirements, infrastructure constraints, and risk profiles, and their source texts may differ. The identity of the system, however, will be defined by global actor IDs, transactional contracts, and graph invariants rather than by a particular program listing.

12. The Actor Graph Is More Than Another Programming Language

In the broad mathematical sense, any formal means of describing computation can be called a language. In that sense, the Actor Graph may also be classified as a formal graph-based programming language.

Such a definition obscures the central change.

A traditional program primarily specifies a method of implementation: the functions, procedures, data, and instructions that must be organized to produce a result. Even in a declarative language, the domain model usually remains distributed between the program and its surrounding artifacts.

The Actor Graph describes a model of relationships in action. It brings together globally identified participants, their `accounts` and `events`, transactions, processes, rights, constraints, jurisdictional boundaries, and a history of change. A relationship is more than an arrow: it may itself be an actor. A nested graph does not lose identity when it is collapsed into a single actor.

The graph simultaneously serves as architecture, an executable domain model, a digital twin, and a source of multiple implementations.

The change reaches beyond a new syntax for expressing familiar programs. The source of operational truth for the system changes.

In the traditional arrangement, the model is part of the program or documentation attached

to it. In the new arrangement, the program becomes one possible implementation of an accepted executable model. For execution, the model is primary and code is secondary.

13. The Disappearance of the Program as a Separate Product

A conventional program is created as a relatively finished product. It is designed, written, tested, released, and subsequently updated through separate versions.

An executable model need not have a final form. It develops together with the system it represents. Real transactions change actors' `accounts`. Their `events` preserve causal history. Telemetry returns execution results. AI detects discrepancies, proposes new relationships, and helps restructure individual parts of the graph. Accepted changes are verified again and become part of the executable model.

The gap between design, programming, and operations begins to narrow.

The digital twin ceases to be a representation of the running system. For the part of the loop under control, it becomes the system's current executable model; for the external part, it becomes a model of identities, protocols, declared states, evidence, and transactional relationships.

Within this construction, the Actor Graph can be understood as the genotype of the system and a particular implementation as its phenotype. A genotype is not a finished phenotype, nor does it eliminate the influence of the environment, but it preserves identity and the rules of development. AI becomes a mechanism of construction and transformation, while the execution environment is where the model encounters reality.

Code is no longer a unique artifact that must be preserved at any cost. It can be rebuilt for another infrastructure without losing the identity of the system itself.

14. Complexity Does Not Disappear

Here we encounter Frederick Brooks's central objection. In *No Silver Bullet*, presented in 1986 and published in 1987, Brooks divided software complexity into accidental and essential complexity. Accidental complexity is generated by the mode of representation and by tools: machine instructions, inconvenient languages, manual assembly, and other technical obstacles. Essential complexity belongs to the system itself—to its rules, exceptions, interactions, contradictions, and consequences [3].

High-level languages and compilers radically reduced the accidental complexity involved in translating intent into machine instructions. AI can reduce the next portion of accidental complexity: the manual translation of a human description into a formal artifact. The Actor Graph can reduce the cost of recoding one model for different implementations.

Neither AI nor the graph eliminates essential complexity. They do not automatically determine who counts as a good customer, who has decision authority, what should happen when norms conflict, which risks are acceptable, or which consequences society is prepared to accept.

The proposed construction is therefore neither a "silver bullet" nor a promise to make complex systems simple. Its claim is more modest and, at the same time, stronger: essential complexity should be moved into a form in which it is visible, executable, and verifiable.

The graph does not eliminate domain complexity. It makes that complexity explicit as actors, `accounts`, `events`, edge actors, transactions, rights, boundaries, and invariants. A mistaken representation of that complexity can be specified with precision and executed with equal precision, which is why verification remains mandatory. At least the dispute is no longer about what is

hidden across thousands of implementation files, but about an explicit model of how the system is supposed to act.

If the production of a particular implementation becomes almost free, the principal scarcity is no longer code, but an accurate model of reality.

15. What Remains for Humans

The disappearance of manual coding does not imply the disappearance of engineering work. On the contrary, the automation of implementation exposes tasks that were previously concealed by the difficulty of writing code.

It also exposes the political dimension of formalization.

In *Do Categories Have Politics?*, Lucy Suchman criticized the language/action perspective precisely because categorizing intentions and permitted speech acts can turn a coordination system into a mechanism for disciplining and controlling participants in an organization [15]. This criticism applies directly to the Actor Graph. If the graph begins to determine not only which changes of state the system recognizes, but also what a person is permitted to say, think, or do, the executable model becomes a machine of organizational coercion.

Winograd responded that an explicit structure of communication should not be confused with an exhaustive model of human behavior. Its function is closer to that of a shared record-keeping procedure: to give a distributed organization a consistent format for recording and coordination where ambiguity cannot be resolved each time through personal conversation [16]. This response matters. A large system cannot, in fact, exist without formal rules for recognizing events, rights, and obligations. Yet it does not remove the political question. Record-keeping categories also determine what becomes visible, recognized, and binding.

The boundary must therefore be stated firmly: **the graph types the system's transitions, not permissible human speech**. It records what change the system is prepared to recognize as valid, who asserted it, when, on what grounds, with what evidence, and under which jurisdiction. A person may act and speak outside the graph; the absence of an accepted transaction means the absence of a system-recognized change, not proof that the event did not occur in reality.

Every category, role, prohibition, and right in the graph must therefore have an author, a rationale, a version, a scope, and a procedure for challenge. Otherwise, the precision of the system will be achieved at the cost of excluding everything that did not fit its classification.

Humans will still have to determine why the system exists, who participates in it, which goals are admissible, where its boundaries lie, and who is responsible for what. They will have to decide which changes are prohibited, what counts as an error, whose state is authoritative, which evidence can be trusted, and which consequences are acceptable.

The value of domain knowledge, architecture, global identity, invariants, reference cases, simulations, historical replay, challenge procedures, and acceptance criteria increases.

The central engineering question changes:

from "Can we program this?" to "How can we prove that the system we have built corresponds to the intent, to reality, and to the established boundaries—and that those boundaries are themselves legitimate?"

Verification will have to address more than lines of code. It will have to address the structure of the graph, the criteria by which actors are distinguished, permitted transitions, participants' rights, edge-actor contracts, transaction integrity, ledger continuity, the provenance of categories, and the

consequences of change.

16. After Code

Programming languages were a great accelerator. They freed humans from the need to think in machine instructions and radically increased the productivity of software development. They are not, however, the final form of interaction between humans and computing systems.

Winograd raised the question of moving beyond language-centered programming in 1979, and subsequent model-driven and workflow approaches demonstrated that a model can be both formal and executable. They also revealed two limits: a separate model drifts away from the running system, while an executable model of an organization inevitably embeds someone's categories and rules. The proposed transition therefore changes the status of code rather than merely repeating workflow at a larger scale: the universal Actor Graph becomes the accepted operational model, and its categories acquire explicit provenance and a procedure for challenge.

AI adds the missing interface. It makes it possible to extract the presumed structure of intent from natural language and heterogeneous forms of human context. It does not make the model true, but it radically reduces the cost of constructing, verifying, explaining, and changing it.

Torvalds is right: today, AI does extend the chain of tools that began with assembly language and the compiler. The next stage goes beyond having AI write more code faster than a human. That remains a transitional phase.

The real change begins when human intent no longer has to be translated manually into program text, and the precision of the system is stored not in an implementation listing, but in a verifiable executable graph.

The Actor Graph of the system becomes the source of operational truth. It does not exhaust reality or acquire a monopoly over it; it authoritatively records which identities, states, events, evidence, and transitions the system itself recognizes. Natural language serves as the means of expressing intent. AI helps transform that intent into a model. Verification separates a proposed interpretation from an accepted system. The execution environment turns the model into actual events and transactions. Procedures for changing and challenging categories prevent formal precision from becoming a monopoly over reality.

Machine code will remain because the machine executes it. Programming languages may disappear because their primary addressee was the human.

References

[1] Winograd, T. "Beyond Programming Languages." *Communications of the ACM*, 22(7), 1979, pp. 391–401. DOI: 10.1145/359131.359133.

[2] Naur, P. "Programming as Theory Building." *Microprocessing and Microprogramming*, 15(5), 1985, pp. 253–261. DOI: 10.1016/0165-6074(85)90032-8.

[3] Brooks, F. P., Jr. "No Silver Bullet—Essence and Accidents of Software Engineering." *Computer*, 20(4), 1987, pp. 10–19. DOI: 10.1109/MC.1987.1663532. The first version was presented in 1986.

[4] Object Management Group. *MDA Guide, Revision 2.0*. OMG, 2014.

[5] Mellor, S. J.; Balcer, M. J. *Executable UML: A Foundation for Model-Driven Architecture*. Addison-Wesley, 2002.

[6] Torvalds, L.; Hohndel, D. "Linus Torvalds in Conversation with Dirk Hohndel." Keynote, Open Source Summit North America, Minneapolis, 20 May 2026. Official event page: `https://events.linuxfoundation.org/archive/2026/open-source-summit-north-america/`. Video recording with the cited passage: `https://youtu.be/fi29pfLcW4I?t=1680`.

[7] Vityaz, A. *Where Does an Actor End? Half a Century of the Actor Model—from the "Big Idea of Messaging" to an Open Transaction Graph of the World in Action*. Unpublished position paper, version 3.9.4, July 2026; preprint in preparation.

[8] Vityaz, A. *Active Transaction Graphs: A Formal Framework for Transactional Interactive Systems*. Zenodo, 2026. DOI: `10.5281/zenodo.20747873`.

[9] Winograd, T. "Introduction to the Language/Action Perspective." *ACM Transactions on Office Information Systems*, 6(2), 1988, pp. 83–86.

[10] Flores, F.; Graves, M.; Hartfield, B.; Winograd, T. "Computer Systems and the Design of Organizational Interaction." *ACM Transactions on Office Information Systems*, 6(2), 1988, pp. 153–172. DOI: `10.1145/45941.45943`.

[11] Medina-Mora, R.; Winograd, T.; Flores, R.; Flores, F. "The ActionWorkflow Approach to Workflow Management Technology." *Proceedings of CSCW '92*, 1992, pp. 281–288. DOI: `10.1145/143457.143530`. Extended journal version: *The Information Society*, 9(4), 1993, pp. 391–404. DOI: `10.1080/01972243.1993.9960152`.

[12] Action Technologies, Inc. U.S. Patents 5,208,748 and 5,216,603, "Method and Apparatus for Structuring and Managing Human Communications by Explicitly Defining the Types of Communications Permitted Between Participants," 1993; U.S. Patent 5,734,837, "Method and Apparatus for Building Business Process Applications in Terms of Its Workflows," 1998.

[13] Hollingsworth, D. *The Workflow Reference Model*. Workflow Management Coalition, Document TC00-1003, Issue 1.1, 29 November 1994.

[14] Object Management Group. *Business Process Model and Notation (BPMN), Version 2.0.2*. OMG Document formal/13-12-09, January 2014.

[15] Suchman, L. "Do Categories Have Politics? The Language/Action Perspective Reconsidered." *Computer Supported Cooperative Work*, 2(3), 1994, pp. 177–190. DOI: `10.1007/BF00749015`.

[16] Winograd, T. "Categories, Disciplines, and Social Coordination." *Computer Supported Cooperative Work*, 2(3), 1994, pp. 191–197. DOI: `10.1007/BF00749016`.